ĐẠI HỌC QUỐC GIA TPHCM

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN CÔNG NGHỆ TRI THỨC

Đồ án 1: Triển khai hệ thống CI

Đề tài: Triển khai hệ thống CI cho dự án YAS

Môn học: DevOps

Sinh viên thực hiện:

Nguyễn Lê Thế Vinh (MSSV1)

Phạm Quang Vinh (MSSV2)

Lê Xuân Trí (MSSV3)

Giáo viên hướng dẫn:

[Tên giảng viên]

Ngày 3 tháng 5 năm 2026

Mục lục

1 Tổng quan dự án

1.1 Giới thiệu YAS

YAS (*Yet Another Shop*) là một dự án mã nguồn mở được phát triển bởi NashTech Garage, nhằm mục đích xây dựng một ứng dụng thương mại điện tử microservice điển hình bằng Java/Spring Boot.

Dự án sử dụng kiến trúc **monorepo** — tất cả các service được quản lý trong một repository duy nhất, bao gồm hơn 19 module:

- **Business Services**: media, product, cart, order, rating, customer, location, inventory, tax, search, payment, promotion, recommendation, delivery, webhook, sampledata
- **BFF (Backend-for-Frontend)**: storefront-bff, backoffice-bff
- **Shared Library**: common-library

1.2 Mục tiêu đồ án

Xây dựng hệ thống **Continuous Integration (CI)** cho dự án YAS bằng Jenkins, đáp ứng các yêu cầu:

1. Sử dụng Jenkins làm CI server
2. Fork repo và cấu hình branch protection
3. Pipeline có ít nhất 2 phase: test và build
4. Monorepo path filter — chỉ build/test service thay đổi
5. Coverage gate $\geq$ 70%
6. Security scanning: Gitleaks, SonarQube, Snyk

1.3 Công nghệ sử dụng

Bảng 1: Công nghệ và công cụ sử dụng

Thành phần	**Công nghệ**	**Phiên bản**
Ngôn ngữ	Java	21
Framework	Spring Boot	3.2
Build tool	Maven	Latest LTS
CI Server	Jenkins	2.555.1
Code hosting	GitHub	—
JDK (Jenkins)	OpenJDK	25
Secret scanning	Gitleaks	8.18.4
Code quality	SonarQube	Self-hosted
Dependency scan	Snyk	CLI
Coverage tool	JaCoCo	0.8.14
Infrastructure	AWS EC2	—

1.4 Thông tin repository

- **Repository gốc**: https://github.com/nashtech-garage/yas
- **Repository nhóm (fork)**: https://github.com/tzin1401/yas
- **Jenkins**: http://3.27.92.213:8080
- **SonarQube**: http://3.27.92.213:9000

2 Kiến trúc hệ thống CI

2.1 Tổng quan kiến trúc

Hệ thống CI được thiết kế với luồng hoạt động như sau:

1. Developer tạo branch mới, viết code và push lên GitHub
2. Developer tạo Pull Request (PR) vào branch `main`
3. GitHub Webhook tự động thông báo Jenkins
4. Jenkins Multibranch Pipeline tự động phát hiện và chạy pipeline
5. Pipeline thực thi 8 stages tuần tự
6. Kết quả CI được báo về GitHub qua commit status
7. PR chỉ được merge khi CI pass và có đủ 2 reviewer approve

2.2 Sơ đồ luồng CI

Luồng CI pipeline bao gồm 8 stages chính:

Bảng 2: 8 Stages của CI Pipeline

Stage	Tên	Mô tả
1	Checkout	Clone code, fetch origin/main
2	Detect Changed Modules	Xác định modules thay đổi (monorepo filter)
3	Gitleaks – Secret Scan	Quét secrets/API keys rò rỉ
4	Test	Chạy unit tests + JaCoCo coverage report
5	Coverage Gate	Kiểm tra coverage $\geq 70\%$
6	Build	Build các modules thay đổi
7	SonarQube – Analysis	Phân tích chất lượng code
8	Snyk – Dependency Scan	Quét lỗ hổng dependencies

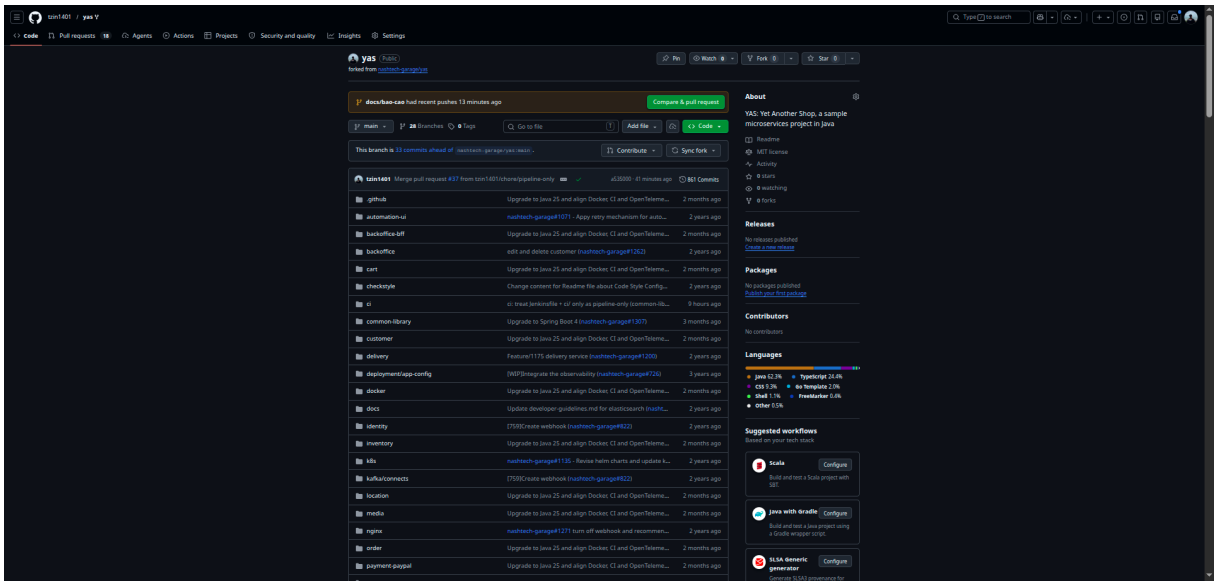
2.3 Hạ tầng và Kiến trúc Build phân tán (Master-Agent)

- **Jenkins Server (Master node)**: Đóng vai trò làm máy chủ trung tâm (Orchestrator). Nhiệm vụ duy nhất là tiếp nhận tín hiệu từ GitHub Webhook, lập lịch, quản lý cấu hình Pipeline, lưu trữ lịch sử build và phân phối tác vụ xuống các Agent.
- **SonarQube Server**: Triển khai độc lập để phân tích chất lượng code của toàn bộ dự án.
- **Distributed Agents (Cộng tác viên build)**: Toàn bộ quá trình checkout code, build, unit test và phân tích bảo mật được giao hoàn toàn cho 3 Agent thực thi song song:
  - **Agent-Tri**: Phụ trách xử lý và thực thi các build job.
  - **Agent-QVinh**: Phụ trách xử lý và thực thi các build job.
  - **Agent-TVinh**: Phụ trách xử lý và thực thi các build job.
- **Lợi ích**: Phân tán tải công việc giúp ngăn chặn tình trạng thắt nút cổ chai (bottleneck) tại Master node, đồng thời giúp tăng tốc thời gian hoàn thành pipeline lên gấp nhiều lần.

3 Cấu hình GitHub

3.1 Fork Repository

Nhóm đã fork repository gốc `nashtech-garage/yas` về tài khoản `tzin1401/yas`. Tất cả thành viên được thêm làm collaborator với quyền write.

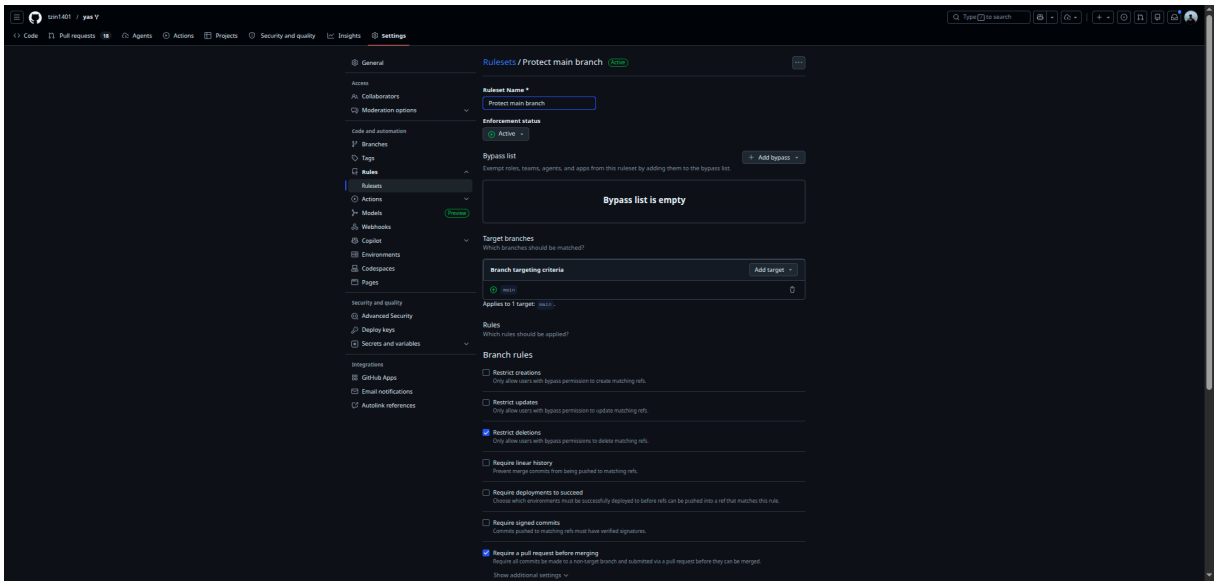


Hình 1: Giao diện repository tzin1401/yas

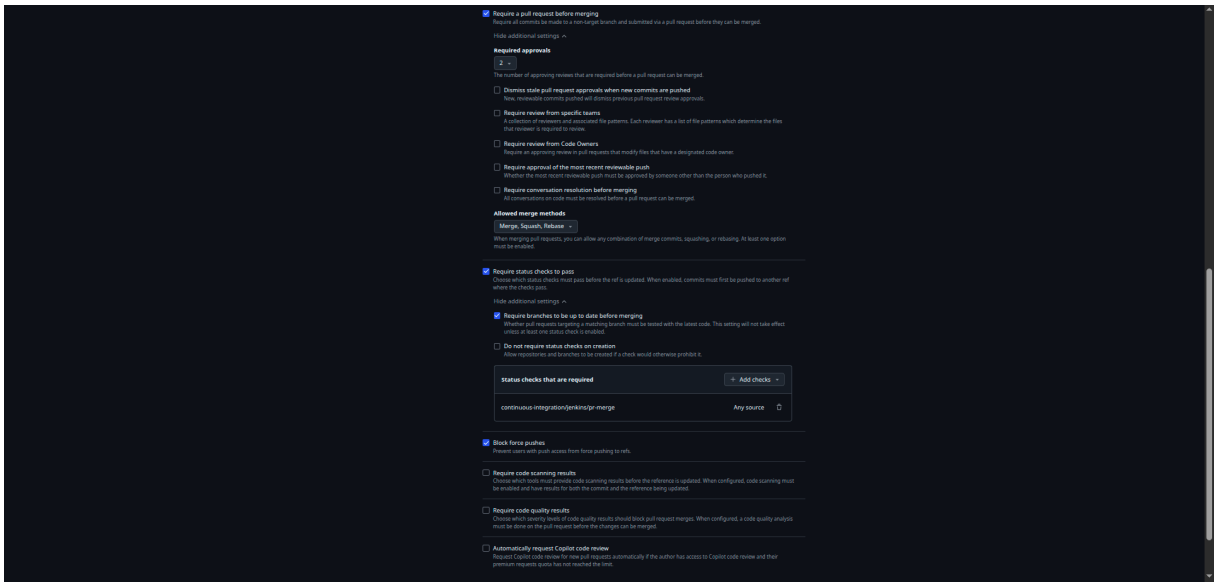
3.2 Branch Protection Rules (GitHub Rulesets)

Để đảm bảo chất lượng code, nhóm đã cấu hình GitHub Rulesets cho branch `main` với các quy tắc:

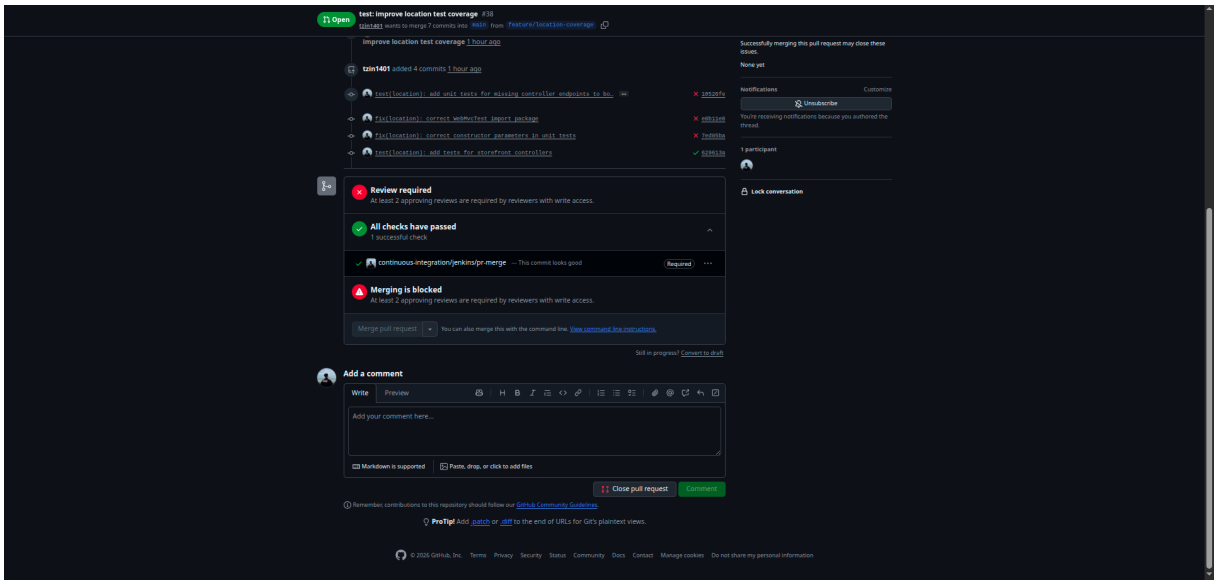
1. **Không cho phép push trực tiếp**: Mọi thay đổi phải thông qua Pull Request
2. **Yêu cầu tối thiểu 2 reviewer approve**: Đảm bảo code review đầy đủ
3. **Required status check**: CI pipeline phải pass (`continuous-integration/jenkins/pr-head`)
4. **Không cho phép force push**: Bảo vệ lịch sử commit



Hình 2: Cấu hình Branch Protection Rules cho nhánh main (Phần 1)



Hình 3: Cấu hình Branch Protection Rules cho nhánh main (Phần 2)

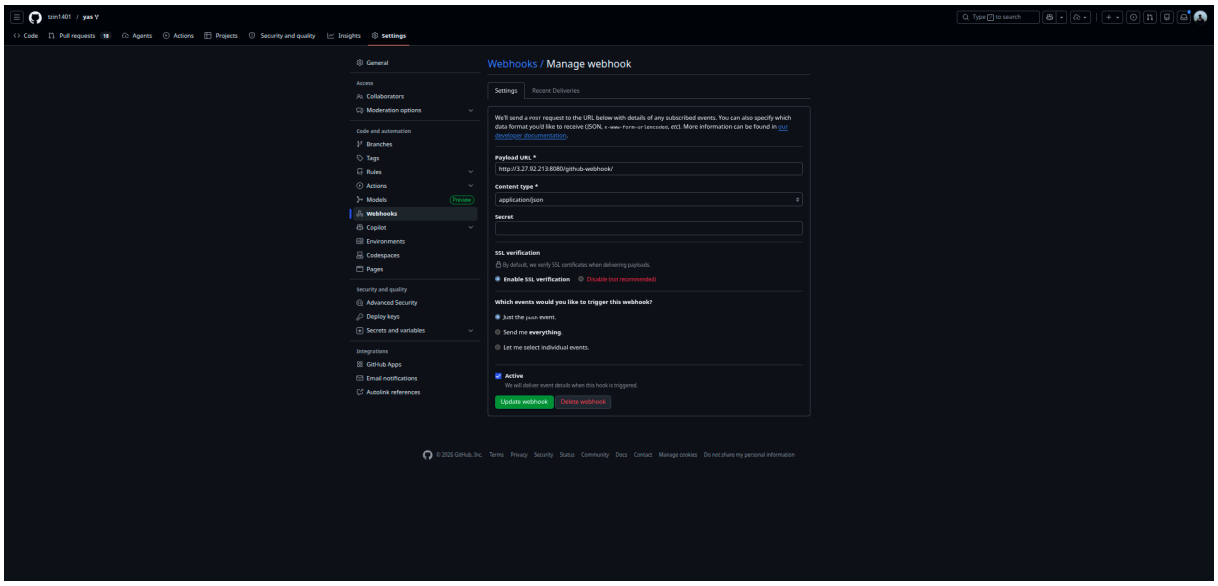


Hình 4: Yêu cầu tối thiểu 2 approvals và CI pass trên Pull Request

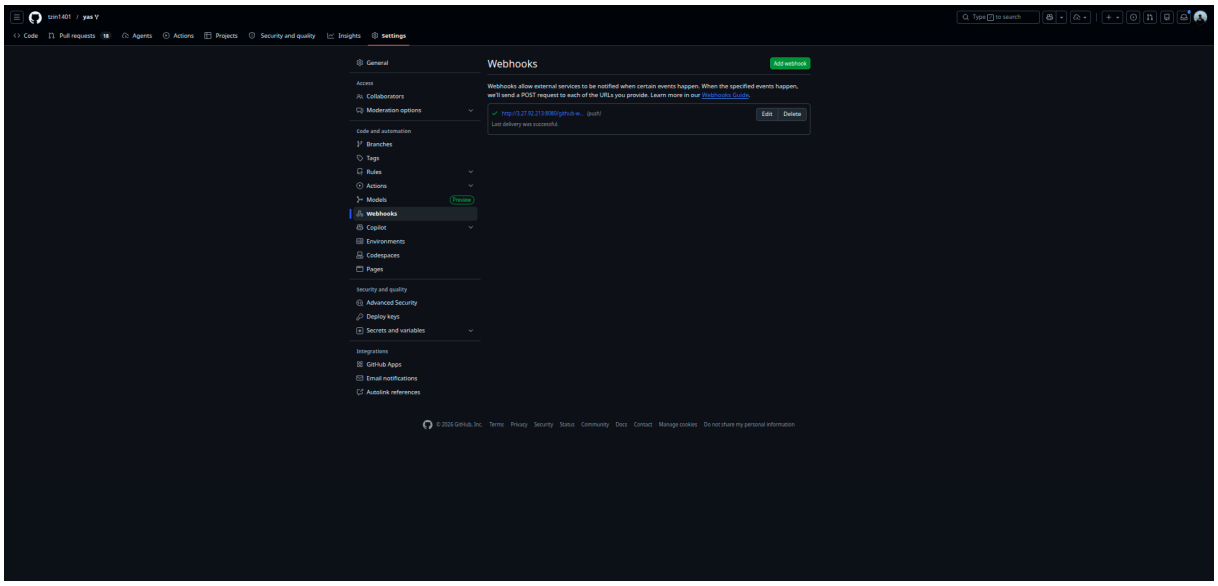
3.3 Webhook GitHub → Jenkins

Cấu hình webhook để GitHub tự động thông báo Jenkins khi có push hoặc PR:

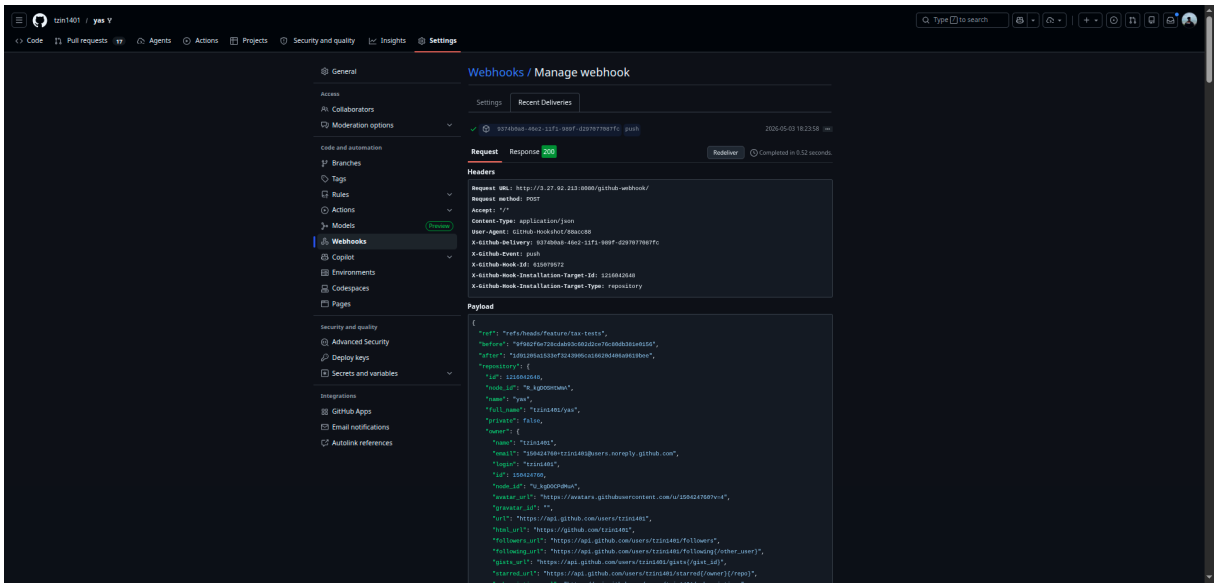
- **Payload URL**: `http://3.27.92.213:8080/github-webhook/`
- **Content type**: `application/json`
- **Events**: Push events, Pull request events



Hình 5: Cấu hình Webhook trỏ tới Jenkins (Phần 1)



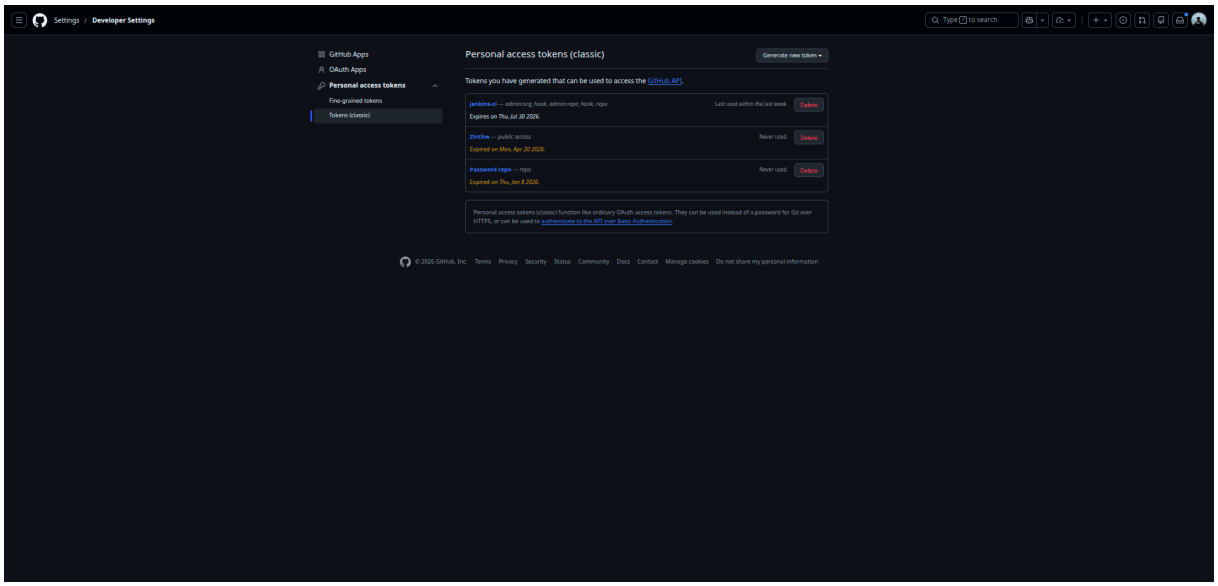
Hình 6: Cấu hình Webhook trỏ tới Jenkins (Phần 2)



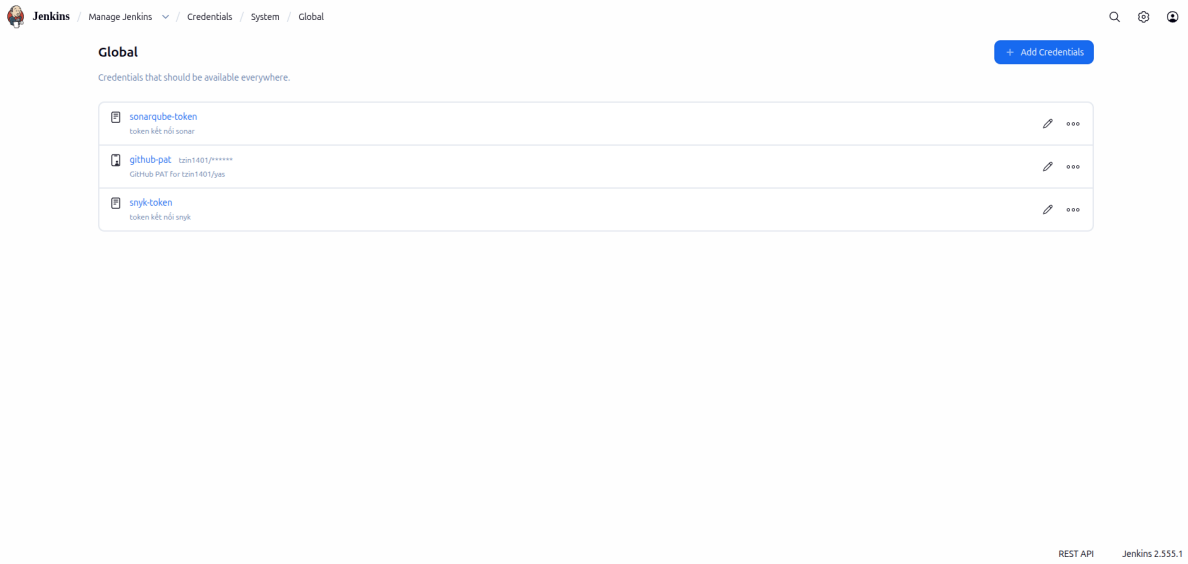
Hình 7: Lịch sử gửi Webhook thành công (HTTP 200)

3.4 Cấp quyền truy cập cho Jenkins

Để Jenkins có thể truy cập và quét các nhánh/PR tự động, nhóm đã khởi tạo Personal Access Token trên GitHub và cấp quyền.



Hình 8: Personal Access Token dành cho Jenkins



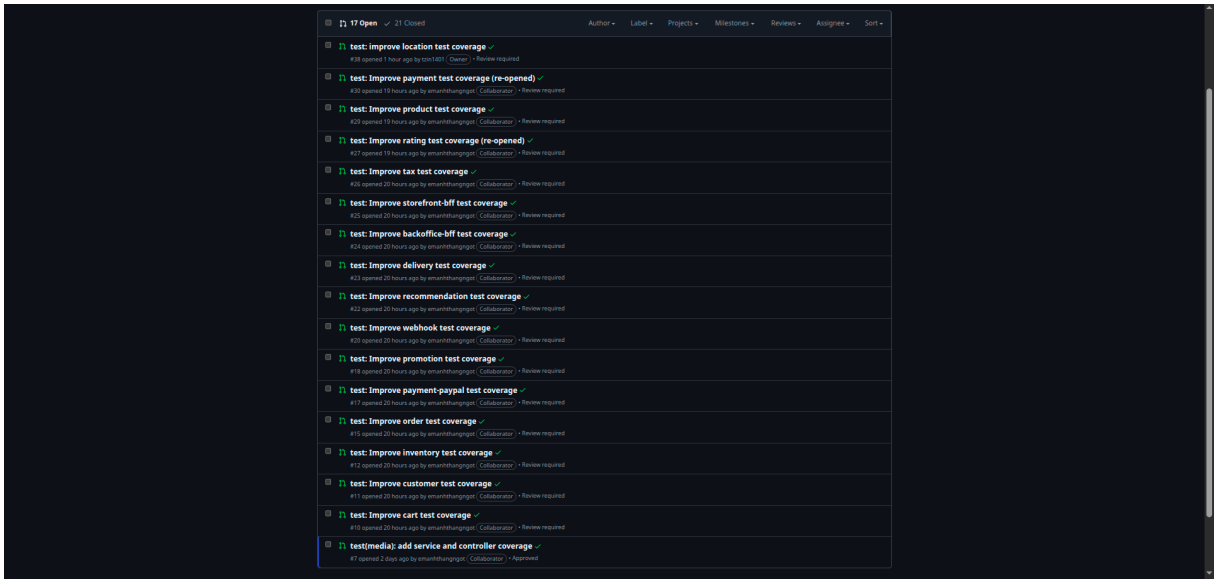
Hình 9: Lưu trữ Token trong Jenkins Credentials

3.5 Quy trình làm việc với Git

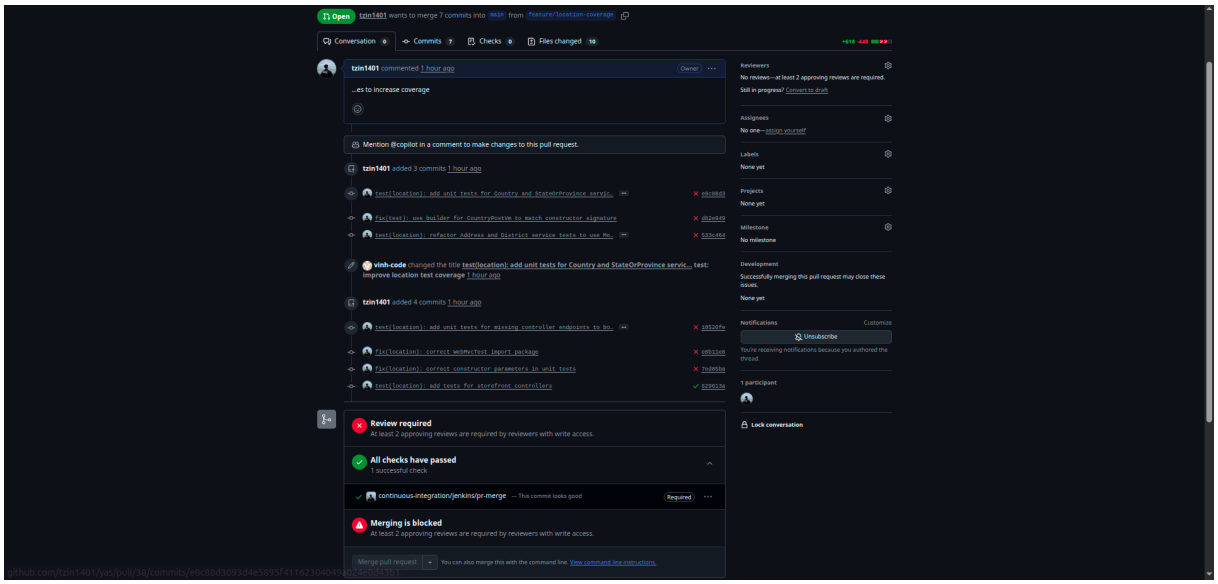
Nhóm tuân theo quy trình Git Flow đơn giản:

1. Tạo branch mới từ `main` (ví dụ: `feature/media-tests`, `ci/setup-jenkins-pipeline`)
2. Phát triển và commit code trên branch
3. Push branch lên GitHub

4. Tạo Pull Request vào `main`
5. Jenkins tự động chạy CI pipeline
6. 2 thành viên khác review và approve
7. Merge PR khi CI pass và đủ approvals



Hình 10: Danh sách các Pull Request theo quy trình



Hình 11: Quá trình trao đổi và review trên Pull Request

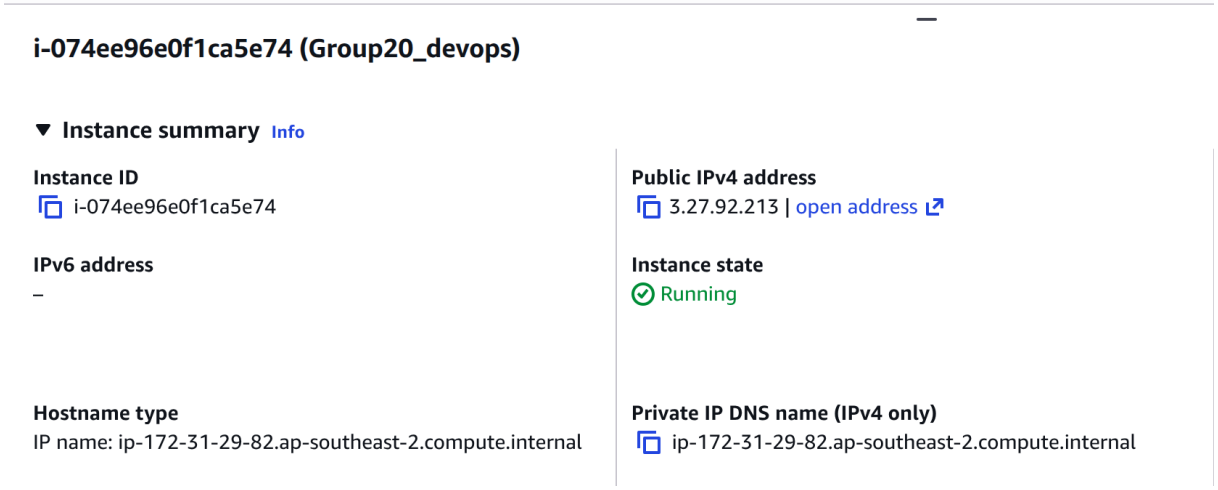
4 Cấu hình máy chủ và Môi trường hệ thống

Để vận hành quy trình CI/CD hoàn chỉnh, nhóm đã triển khai một máy chủ riêng biệt trên nền tảng điện toán đám mây.

4.1 Khởi tạo máy chủ trên AWS (EC2)

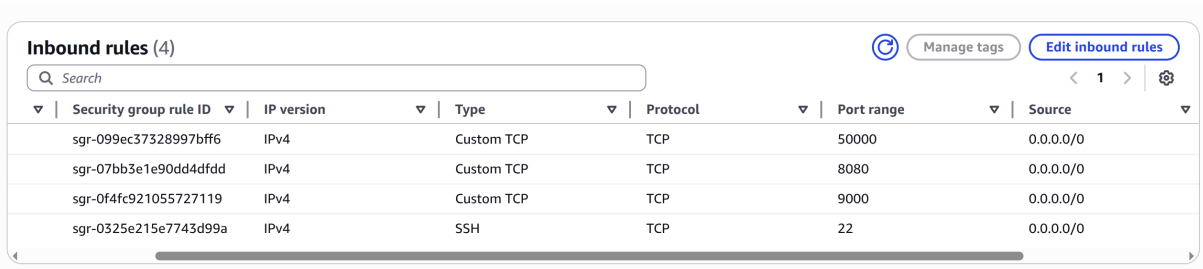
Nhóm sử dụng dịch vụ Amazon EC2 để tạo máy chủ với các thông số sau:

- **Hệ điều hành**: Ubuntu Server 24.04 LTS (Hỗ trợ tốt nhất cho Docker)
- **Instance Type**: `t3.small` (2 vCPU, 2GB RAM).
- **Bộ nhớ mở rộng (Swap Memory)**: Cấu hình 4GB Swap để tránh tình trạng tràn bộ nhớ (Out-of-memory) khi SonarQube và Jenkins chạy đồng thời.



Hình 12: Danh sách các máy chủ EC2 phục vụ hệ thống CI/CD

Trên AWS Console, nhóm tạo một Security Group và mở các cổng mạng thiết yếu (22, 8080, 9000, 50000).



Hình 13: Thiết lập quy tắc Inbound rules của Security Group

4.2 Cài đặt và Quản trị môi trường Docker

Dự án sử dụng công nghệ Container để cô lập môi trường và dễ dàng quản lý dịch vụ.

Cài đặt Docker Engine: Nhóm tiến hành cài đặt các gói `docker.io` và `docker-compose` trên nền tảng Ubuntu.

```
# Cap nhat he thong
sudo apt update && sudo apt upgrade -y

# Cai dat Docker Engine
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker

# Cap quyen chay Docker khong can sudo
sudo usermod -aG docker $USER

# Cai dat Docker Compose
sudo apt install docker-compose -y
```

Listing 1: Cài đặt Docker trên Ubuntu Server

Thiết lập cấu hình triển khai (Docker Compose): Thay vì cài đặt rời rạc, dự án sử dụng Docker Compose để quản lý tập trung Jenkins và SonarQube.

```
version: '3.8'
services:
  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    restart: always

  sonarqube:
```

```
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      - SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true
    volumes:
      - sonarqube_data:/opt/sonarqube/data
    restart: always

volumes:
  jenkins_home:
  sonarqube_data:
```

Listing 2: Tệp cấu hình docker-compose.yml tiêu chuẩn

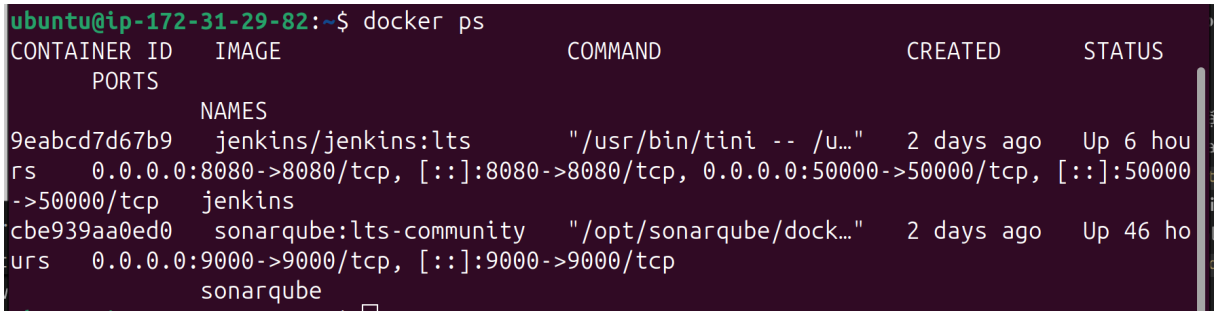
Quản trị bằng Docker CLI: Thay vì cài đặt truyền thống, việc tương tác với dịch vụ được thực hiện qua các lệnh thực thi trực tiếp vào Container.

Lấy mật khẩu Jenkins: Sử dụng lệnh sau để lấy mã unlock hệ thống ban đầu:

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Kiểm tra trạng thái: Sử dụng lệnh sau để xác nhận các container `jenkins` và `sonarqube` đang vận hành ổn định:

```
docker ps
```

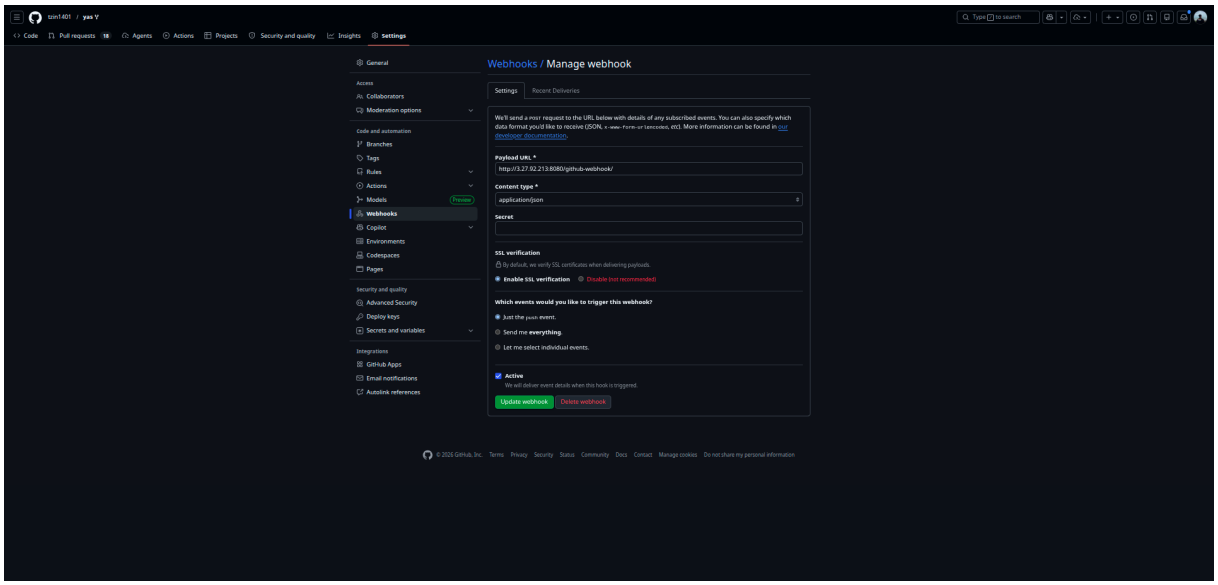


Hình 14: Danh sách các container dịch vụ đang hoạt động ổn định

4.3 Cấu hình tích hợp hệ thống (System Integration)

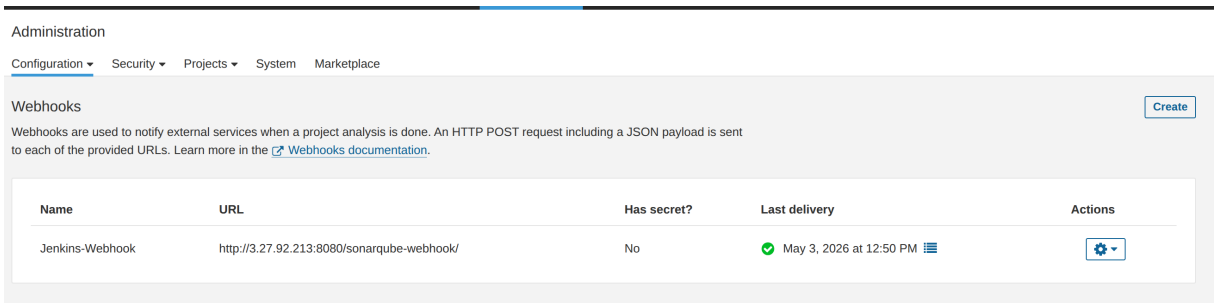
Thiết lập luồng giao tiếp tự động giữa GitHub, Jenkins và SonarQube.

GitHub Webhook: Cấu hình Webhook trên Repository để gửi tín hiệu tới Jenkins mỗi khi có sự kiện Push hoặc Pull Request.



Hình 15: URL Webhook trên GitHub có dấu tích xanh (Active)

SonarQube Webhook: Cấu hình Webhook ngược từ SonarQube về Jenkins (`http://<IP-Jenkins>:8080/so` để trả kết quả Quality Gate.



Hình 16: Cấu hình Webhook gửi dữ liệu ngược từ SonarQube về lại Jenkins

Quản lý thông tin bảo mật: Thiết lập các ID Credentials trong Jenkins (`sonarqube-token`, `snyk-token`) để kết nối an toàn với các dịch vụ bên thứ ba.

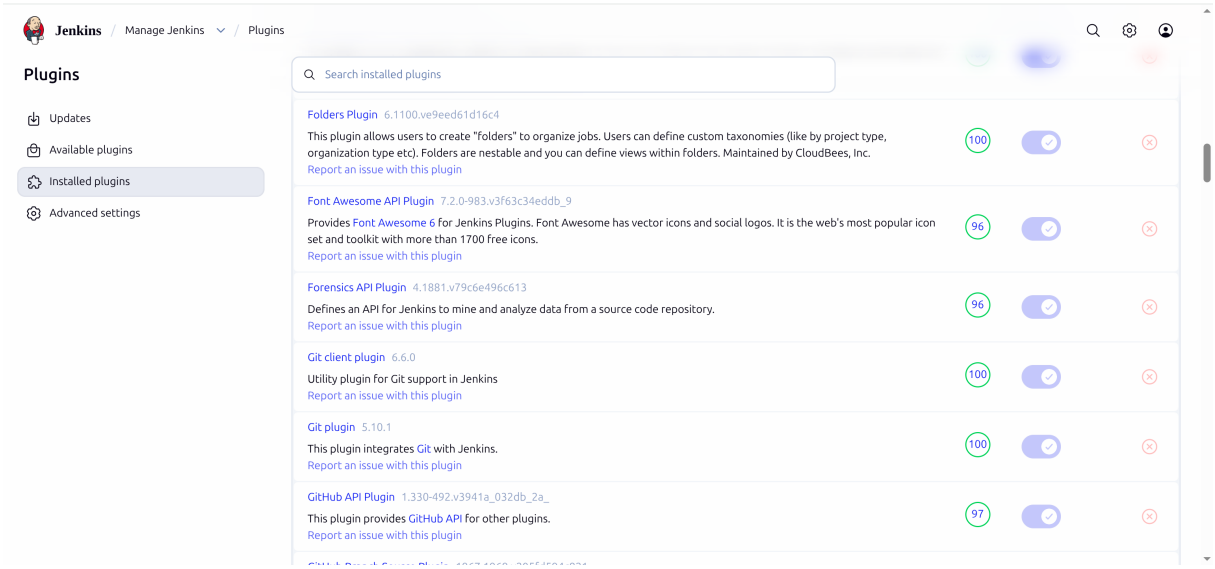
5 Cấu hình Jenkins

5.1 Cài đặt Jenkins trên AWS EC2

Jenkins được cài đặt trên AWS EC2 instance và truy cập qua `http://3.27.92.213:8080`.

Các plugin cần thiết đã cài đặt:

- **GitHub Branch Source Plugin**: Hỗ trợ Multibranch Pipeline với GitHub
- **Pipeline Plugin**: Hỗ trợ Jenkinsfile declarative pipeline
- **JUnit Plugin**: Hiển thị kết quả test
- **Credentials Binding Plugin**: Quản lý secrets (SonarQube token, Snyk token)
- **Timestamps Plugin**: Hiển thị thời gian trong console output

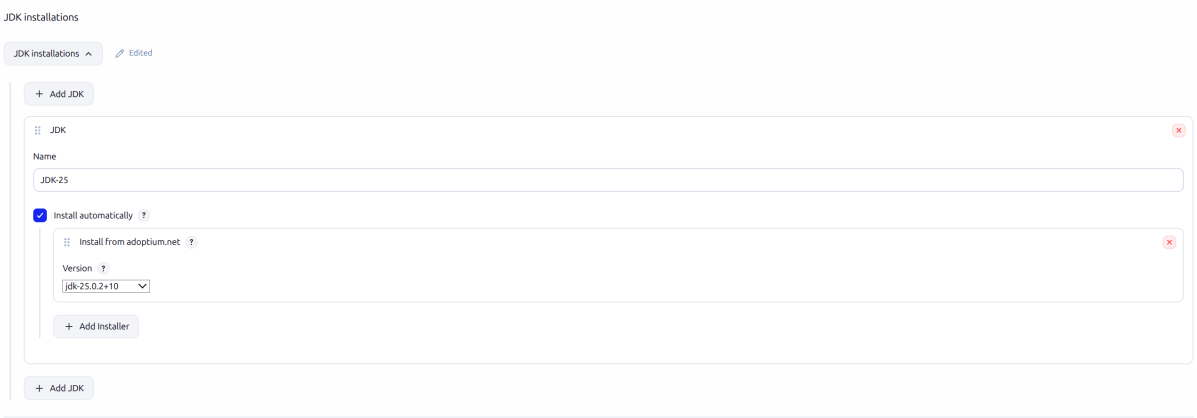


Hình 17: Giao diện Dashboard quản lý Jenkins trên AWS EC2

5.2 Cấu hình Tools

Trong Jenkins Global Tool Configuration:

- **JDK**: Cấu hình JDK-25 (OpenJDK 25)
- **Maven**: Cấu hình Maven (Latest LTS)



Hình 18: Cấu hình JDK-25 trong Jenkins Global Tool Configuration

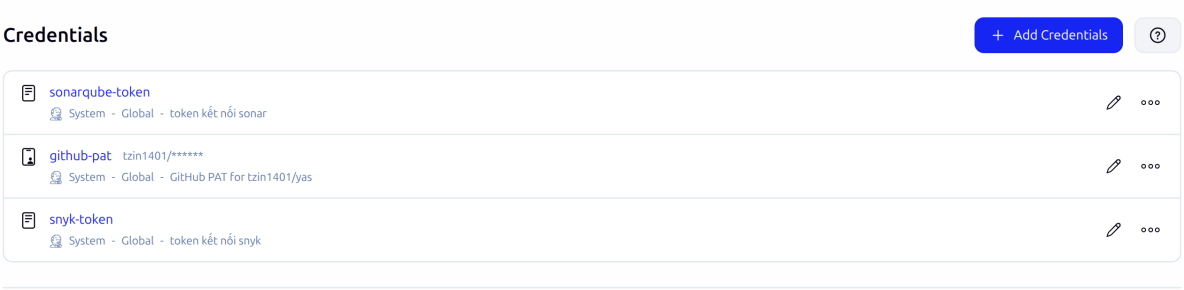
Hình 19: Cấu hình Maven trong Jenkins Global Tool Configuration

5.3 Cấu hình Credentials

Các credentials được lưu trong Jenkins Credentials Store:

Bảng 3: Jenkins Credentials

ID	Loại	Mục đích
`sonarqube-token`	Secret text	Xác thực SonarQube API
`snyk-token`	Secret text	Xác thực Snyk CLI



Hình 20: Danh sách ID Credentials đã được lưu trên Jenkins

5.4 Cấu hình Distributed Build Nodes (Agents)

Để thực thi các tác vụ build và test một cách phân tán, nhóm đã đăng ký 3 Agent vào hệ thống Jenkins Master:

- **Agent-Tri**: Agent phụ trách xử lý và thực thi các build job.
- **Agent-QVinh**: Agent phụ trách xử lý và thực thi các build job.
- **Agent-TVinh**: Agent phụ trách xử lý và thực thi các build job.

Hình 21: Danh sách các Nodes/Agents đã kết nối thành công với Jenkins Master

5.4.1 Thiết lập Jenkins Agent (Node) và Cách kết nối

Trong mô hình Distributed Build, Jenkins Master chỉ điều khiển, còn Agent (Node) là nơi thực sự chạy code.

1.1 Các bước tạo Node trên giao diện Jenkins:

1. Truy cập **Manage Jenkins** → **Nodes** → **New Node**.
2. Đặt tên (vd: `Agent-QVinh`), chọn **Permanent Agent**.

3. **Remote root directory**: Thư mục làm việc trên máy Agent (vd: `/home/ubuntu/jenkins-agent`).
4. **Launch method**: Chọn **Launch agent by connecting it to the controller** (kết nối từ máy Agent đến Master qua JNLP).

1.2 Cách kết nối từ máy Agent đến Node (Master): Sau khi nhấn Save, Jenkins sẽ cung cấp một đoạn mã lệnh. Nhóm tiến hành SSH vào máy Agent và thực hiện:

1. Tải file `agent.jar`:

```
curl -sO http://<IP-Jenkins-Master>:8080/jnlpJars/agent.jar
```

2. Chạy lệnh kết nối:

```
java -jar agent.jar -url http://<IP-Jenkins-Master>:8080/ -secret <
    SECRET_KEY> -name "Agent-QVinh" -workDir "/home/ubuntu/jenkins-agent"
```

3. *Lưu ý*: Để Agent tự khởi chạy lại khi khởi động máy, nhóm cấu hình lệnh này thành một Systemd Service trên Linux.

5.4.2 Cài đặt môi trường thực thi (Tooling) trên Agent

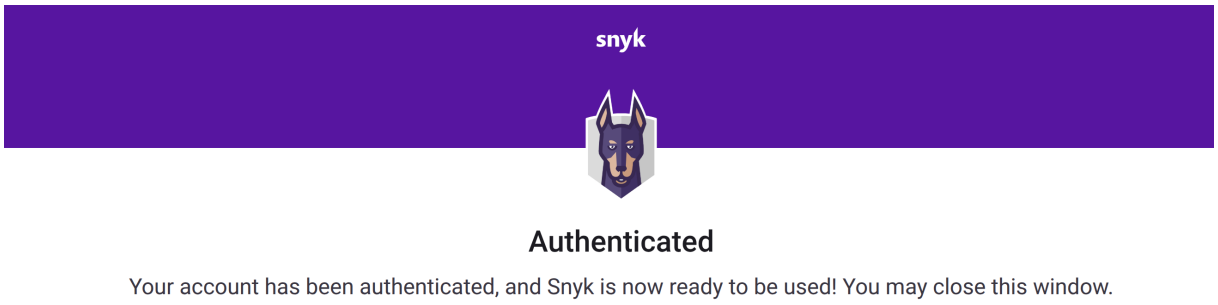
Để chạy được toàn bộ Pipeline "yas", các máy Agent (Node) được cài đặt các công cụ sau:

- **Docker Engine & Docker Compose**: Cài đặt để thực hiện Stage Build Image và quét Snyk Container. Đồng thời, chạy lệnh `sudo usermod -aG docker jenkins` và khởi động lại Agent để user Jenkins có quyền thực thi lệnh Docker.
- **Java Development Kit (JDK)**: Cài đặt phiên bản khớp với dự án (ví dụ JDK 25) để thực thi file `agent.jar` và chạy Maven.
- **Apache Maven**: Cài đặt để chạy lệnh `mvn test`, `mvn dependency:tree` và các lệnh kiểm tra coverage.
- **Git**: Cài đặt để Agent có thể thực hiện Stage Checkout code từ GitHub.

5.4.3 Cấu hình Snyk Auth trên Terminal của Agent

Trong môi trường CI/CD, việc xác thực Agent bằng Token là tối ưu nhất.

1. **Cài đặt Snyk CLI**: Agent cài đặt Snyk thông qua lệnh `npm install -g snyk`.
2. **Xác thực bằng Token**: Trên Terminal của Agent, chạy lệnh: `snyk auth <YOUR_SNYK_API_TOKEN>` hoặc gán biến môi trường `SNYK_TOKEN` để Snyk tự động nhận diện.

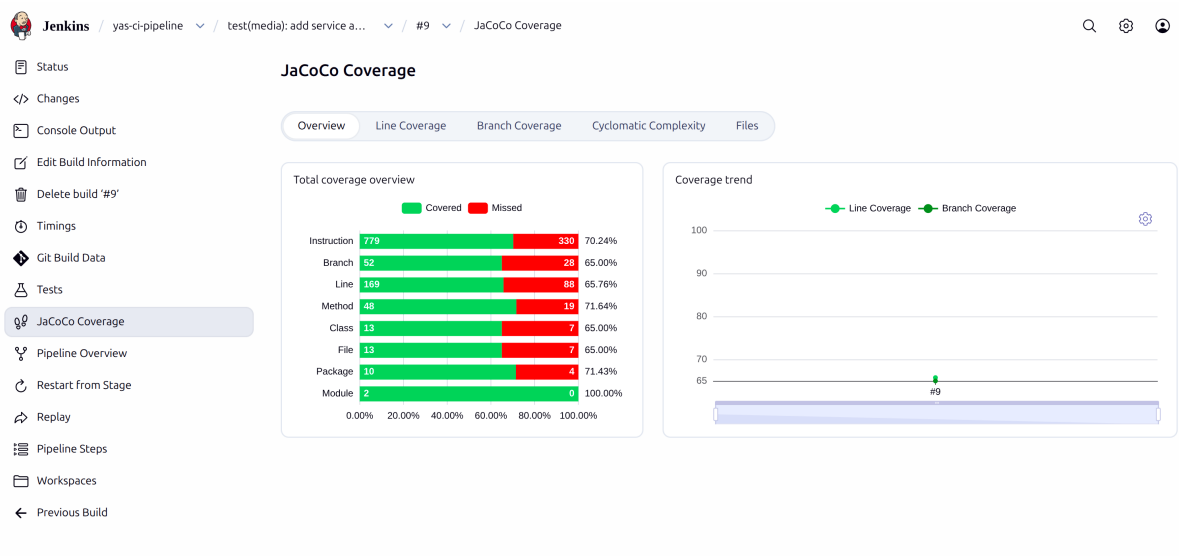


Hình 22: Xác thực tài khoản Snyk thành công trên Agent

5.5 Cấu hình hiển thị Biểu đồ và Thống kê (Plugins)

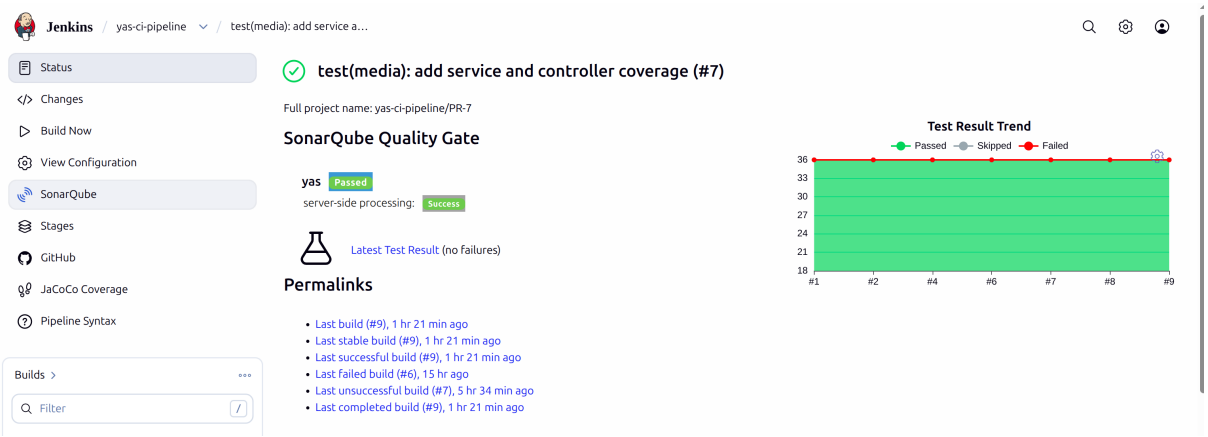
Để Jenkins hiển thị được các tab báo cáo và biểu đồ từ kết quả quét, nhóm đã cấu hình các Plugin sau trên Master:

- **JaCoCo Plugin**: Sau khi chạy `mvn jacoco:report`, plugin này đọc file `.exec` trong thư mục target để hiển thị biểu đồ Coverage Trend ngay tại trang chủ của Job.



Hình 23: Biểu đồ Coverage Trend hiển thị thông qua JaCoCo Plugin

- **SonarQube Scanner Plugin**: Cấu hình Webhook trỏ về `/sonarqube-webhook/` để hiển thị trạng thái Quality Gate (Passed/Failed) trực tiếp trên Pipeline.



Hình 24: Thống kê của SonarQube

- **Snyk Security Plugin**: Cho phép xem chi tiết các lỗ hổng (Vulnerabilities) theo mức độ nghiêm trọng (High, Medium, Low) ngay trong giao diện Jenkins.

5.6 Tạo Multibranch Pipeline Job

Nhóm tạo job `yas-ci-pipeline` với loại **Multibranch Pipeline**:

1. **Branch Sources**: GitHub repository `tzin1401/yas`
2. **Discover branches**: Strategy "Exclude branches that are also filed as PRs" — tránh build trùng lặp khi branch đã có PR
3. **Discover pull requests from origin**: Merging the pull request with the current target branch revision
4. **Build Configuration**: Jenkinsfile tại root của repository
5. **Scan Repository Triggers**: Tự động quét mỗi khi có webhook từ GitHub

img/04-jenkins-config-branch-sources.png

Hình 25: Cấu hình Branch Sources trong Multibranch Pipeline Job

img/05-jenkins-config-jenkinsfile-path.png

Hình 26: Cấu hình đường dẫn Jenkinsfile trong Multibranch Pipeline Job

5.7 Kết quả: Jenkins tracking branches và PRs

Sau khi cấu hình, Jenkins tự động phát hiện và tracking:

- **5 branches**: main, ci/setup-jenkins-pipeline, feature/media-tests, tri/tax-tests, test/ci-path-filter
- **5 pull requests**: PR #3, #4, #5, #6, #7

img/02-jenkins-multibranch-branches.png

Hình 27: Danh sách các nhánh được tự động phát hiện và build

img/03-jenkins-multibranch-prs.png

Hình 28: Danh sách các Pull Request được tự động phát hiện và build

6 Chi tiết Pipeline (Jenkinsfile)

Pipeline được viết bằng **Declarative Pipeline** trong file `Jenkinsfile` tại root của repository. Tổng cộng 298 dòng, 8 stages chính.

6.1 Cấu hình chung

```
pipeline {
```

```
    agent any
    tools {
        maven 'Maven'
        jdk   'JDK-25'
    }
    options {
        timestamps()
        buildDiscarder(logRotator(numToKeepStr: '10'))
        timeout(time: 60, unit: 'MINUTES')
        disableConcurrentBuilds()
    }
    environment {
        COVERAGE_THRESHOLD = '70'
        SONAR_HOST_URL     = 'http://3.27.92.213:9000'
        MAVEN_OPTS         = '-Xmx512m -XX:MaxMetaspaceSize=256m'
    }
}
```

Listing 3: Cấu hình pipeline

Các cấu hình quan trọng:

- `COVERAGE_THRESHOLD = '70'`: Ngưỡng coverage tối thiểu 70%
- `MAVEN_OPTS`: Giới hạn bộ nhớ JVM để phù hợp với EC2
- `disableConcurrentBuilds()`: Tránh xung đột khi nhiều builds chạy cùng lúc

6.2 Stage 1: Checkout

```
stage('Checkout') {
    steps {
        checkout scm
        sh 'git fetch origin main:refs/remotes/origin/main || true'
        sh 'chmod +x ci/detect-changed-modules.sh ci/check-coverage.sh'
    }
}
```

Listing 4: Stage Checkout

Stage này thực hiện:

- Clone source code từ GitHub
- Fetch `origin/main` để có baseline cho việc so sánh thay đổi (fix shallow clone của Jenkins)
- Cấp quyền thực thi cho các CI scripts

6.3 Stage 2: Detect Changed Modules

```
stage('Detect Changed Modules') {
    steps {
        script {
            env.CHANGED_MODULES = sh(
                script: 'ci/detect-changed-modules.sh',
                returnStdout: true
            ).trim()
        }
    }
}
```

Listing 5: Stage Detect Changed Modules

Stage này gọi script `ci/detect-changed-modules.sh` để xác định modules nào thay đổi so với `origin/main`. Chi tiết về script được mô tả ở Mục **??**.

6.4 Stage 3: Gitleaks – Secret Scan

Quét toàn bộ source code tìm secrets/API keys bị rò rỉ. Chi tiết ở Mục **??**.

6.5 Stage 4: Test (JUnit + JaCoCo)

```
stage('Test') {
    steps {
        // ... mvn test jacoco:report ...
    }
    post {
        always {
            junit allowEmptyResults: true,
                  testResults: '**/target/surefire-reports/*.xml'
            recordCoverage(
```

```
                tools: [[parser: 'JACOCO', pattern: '**/target/site/jacoco/jacoco.xml']],
                id: 'jacoco', name: 'JaCoCo Coverage',
                sourceDirectories: [ [path: 'location/src/main/java'], /* ... */ ]
            )
        }
    }
}
```

Listing 6: Stage Test

Stage này:

- Chỉ chạy test cho **modules thay đổi** (`-pl ${module}`)
- Tự động build dependencies (`-am`: also make)
- Skip Integration Tests (`-DskipITs`)
- Tích hợp plugin **recordCoverage** cho phép vẽ biểu đồ Coverage và xem chi tiết từng dòng code đỏ/xanh trực tiếp trên giao diện Jenkins.

6.6 Stage 5: Coverage Gate

```
stage('Coverage Gate') {
    steps {
        script {
            def modules = env.CHANGED_MODULES.split(',')
            def serviceModules = modules.findAll { it != 'common-library' }
            if (serviceModules.isEmpty()) {
                echo ">>> No service modules to check coverage"
            } else {
                for (module in serviceModules) {
                    sh "ci/check-coverage.sh ${module} ${env.COVERAGE_THRESHOLD}"
                }
            }
        }
    }
```

```
}
```

Listing 7: Stage Coverage Gate

Stage này:

- Gọi script `ci/check-coverage.sh` để kiểm tra coverage $\geq 70\%$
- **Exclude common-library**: Đây là shared library với coverage 33% từ upstream, không phải service module
- Script parse JaCoCo XML report, tính LINE coverage percentage
- Pipeline FAIL nếu bất kỳ service module nào có coverage $< 70\%$

6.7 Stage 6: Build

```
stage('Build') {
    steps {
        script {
            def modules = env.CHANGED_MODULES.split(',')
            for (module in modules) {
                sh "mvn -B -ntp -DskipTests -pl ${module} -am package"
            }
        }
    }
}
```

Listing 8: Stage Build

Build chỉ các modules thay đổi, skip tests (đã chạy ở stage 4).

6.8 Stage 7: SonarQube Analysis

Phân tích chất lượng code. Chi tiết ở Mục **??**.

6.9 Stage 8: Snyk Dependency Scan

Quét lỗ hổng bảo mật trong dependencies. Chi tiết ở Mục **??**.

6.10 Post Actions

```
post {
    always { cleanWs() }
    success {
        echo "PIPELINE PASSED SUCCESSFULLY"
    }
    failure {
        echo "PIPELINE FAILED - Check console output"
    }
}
```

Listing 9: Post-build actions

- `cleanWs()`: Dọn workspace sau mỗi build để tiết kiệm disk
- In thông báo kết quả (pass/fail) kèm branch name và build number

7 Monorepo Path Filter

7.1 Vấn đề

YAS là một monorepo chứa hơn 19 modules. Nếu mỗi lần push đều build/test **toàn bộ** modules, thời gian CI sẽ rất lâu và lãng phí tài nguyên. Cần một cơ chế chỉ build/test các modules thực sự thay đổi.

7.2 Giải pháp: detect-changed-modules.sh

Nhóm viết script `ci/detect-changed-modules.sh` (74 dòng) để xác định modules thay đổi so với `origin/main`.

```
#!/usr/bin/env bash
set -euo pipefail

# Danh sach cac module cua monorepo
modules=(
  automation-ui backoffice-bff cart common-library
  customer delivery discount inventory location
```

```
  media order payment product promotion
  rating recommendation sampledata search
  storefront-bff tax webhook
)

# Lay danh sach files thay doi so voi origin/main
changed_files="$(git diff --name-only origin/main...HEAD 2>/dev/null || true)"

# Neu thay doi o root files -> build tat ca modules
if grep -Eq '^(pom\.xml|\.github/|checkstyle/|common-library/|docker/|k8s/|
    scripts/)' \
      <<< "${changed_files}"; then
    printf '%s\n' "${modules[@]}" | paste -sd ','
    exit 0
fi

# Map file -> module
selected_modules=()
for mod in "${modules[@]}"; do
    if grep -q "^${mod}/" <<< "${changed_files}"; then
        selected_modules+=("${mod}")
    fi
done

# Fallback: build common-library
if [[ ${#selected_modules[@]} -eq 0 ]]; then
    selected_modules=("common-library")
fi

printf '%s\n' "${selected_modules[@]}" | paste -sd ','
```

Listing 10: Logic chính của detect-changed-modules.sh

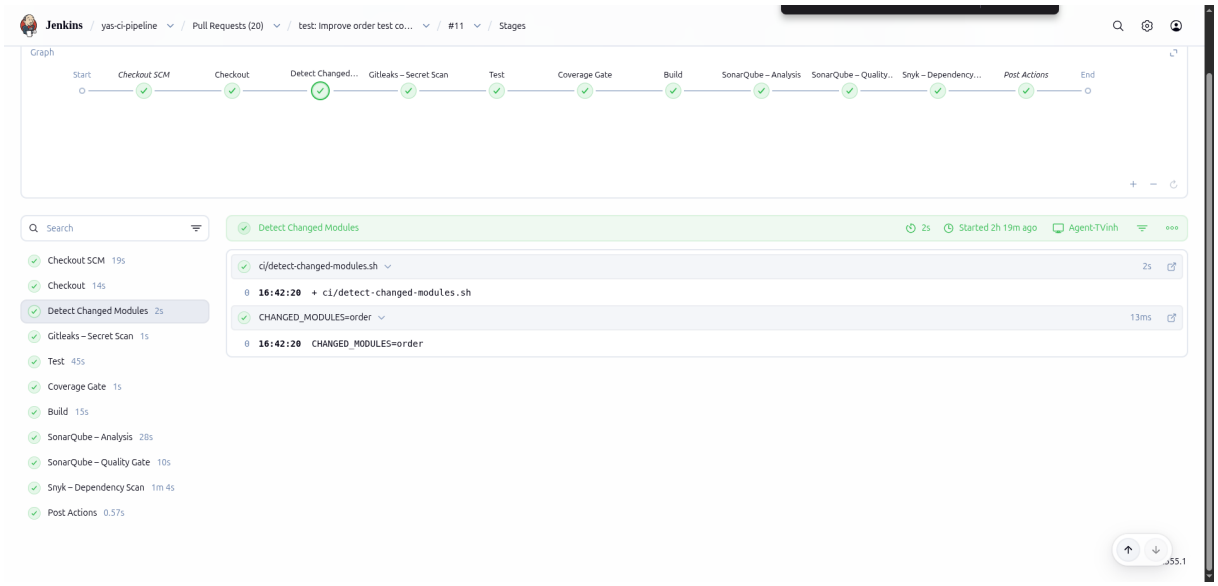
7.3 Chiến lược phát hiện thay đổi

Bảng 4: Quy tắc phát hiện module thay đổi

Files thay đổi	**Modules được build**
`pom.xml` (root)	Tất cả modules
`common-library/*`	Tất cả modules (shared dependency)
`media/src/*`	Chỉ `media`
`tax/src/*`	Chỉ `tax`
`ci/*`, `docs/*`	`common-library` (fallback)

7.4 Kết quả

- Thời gian build giảm đáng kể: thay vì build toàn bộ 19 modules (∼30 phút), chỉ build 1–2 modules (∼3–5 phút)
- Tiết kiệm tài nguyên AWS EC2
- Pipeline vẫn đảm bảo tính chính xác nhờ flag `-am` (also make dependencies)



Hình 29: Console output hiển thị tính năng Path Filter nhận diện các module thay đổi

8 Unit Test và Coverage Gate

8.1 Chiến lược Unit Test

Nhóm viết thêm unit test cho 2 service modules:

- **media**: Test cho MediaService và FileSystemRepository
- **tax**: Test cho TaxService và TaxController

Mỗi PR (Pull Request) chỉ thêm test cho module tương ứng, đảm bảo tính module hóa.

8.2 Chi tiết Test Cases

8.2.1 Module media (PR #7)

Bảng 5: Unit tests cho module media

Test class	**Số tests**	**Kết quả**
MediaServiceUnitTest	20	Pass
FileSystemRepositoryTest	9	Pass
common-library tests (dependency)	7	Pass
Tổng	**36**	**100% Pass**

8.2.2 Module tax (PR #6)

Bảng 6: Unit tests cho module tax

Test class	**Số tests**	**Kết quả**
TaxRateServiceTest	20	Pass
TaxClassServiceTest	10	Pass
TaxRateControllerTest	10	Pass
common-library tests (dependency)	7	Pass
Tổng	**47**	**100% Pass**

8.3 Coverage Gate Script (check-coverage.sh)

Script `ci/check-coverage.sh` kiểm tra LINE coverage từ JaCoCo report:

```
#!/usr/bin/env bash
set -euo pipefail

module="$1"
minimum="$2"
report_path="${module}/target/site/jacoco/jacoco.xml"

# Kiem tra file ton tai
```

```
if [[ ! -f "${report_path}" ]]; then
  echo "[WARN] Coverage report missing - skipping"
  exit 0
fi

# Lay LINE counter cap report-level (dong cuoi cung)
line_counter="$(grep '<counter type="LINE"' "${report_path}" | tail -1)"
covered="$(echo "${line_counter}" | sed 's/.*covered="\([0-9]*\)".*/\1/')"
missed="$(echo "${line_counter}" | sed 's/.*missed="\([0-9]*\)".*/\1/')"

# Tinh phan tram
total=$((covered + missed))
coverage_percent="$(awk -v c="${covered}" -v t="${total}" \
    'BEGIN { printf "%.2f", (c / t) * 100 }')"

echo "Module ${module} line coverage: ${coverage_percent}%"

# So sanh voi nguong
if awk -v actual="${coverage_percent}" -v threshold="${minimum}" \
    'BEGIN { exit !(actual + 0 >= threshold + 0) }'; then
  exit 0
fi

echo "Coverage gate failed: ${coverage_percent}% < ${minimum}%"
exit 1
```

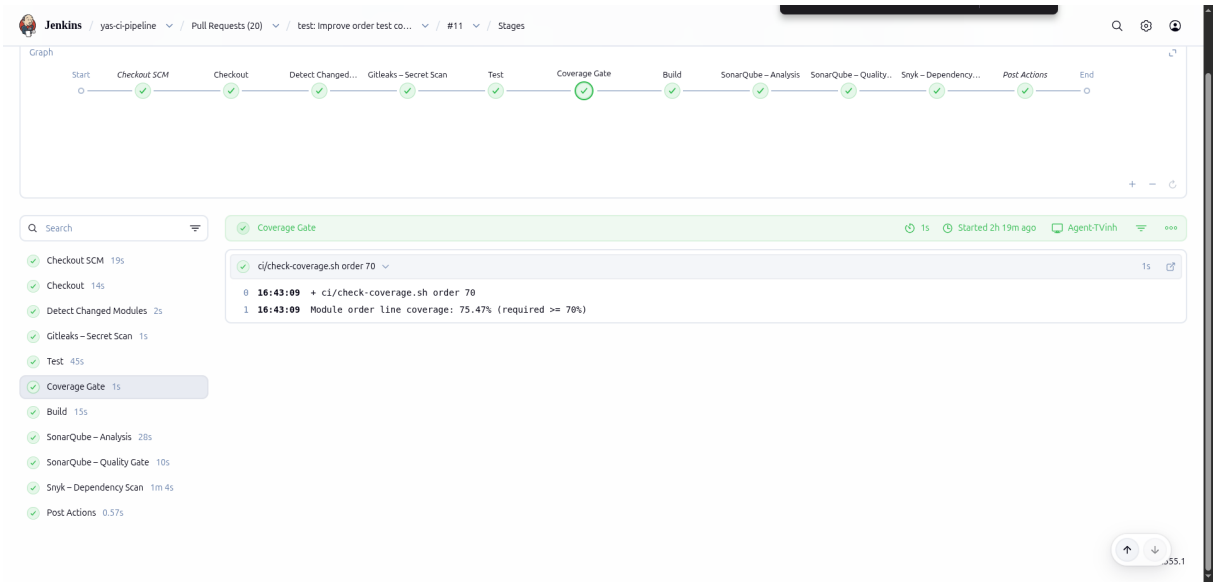
Listing 11: check-coverage.sh - Coverage Gate

8.4 Thiết kế Coverage Gate

- **Ngưỡng**: 70% LINE coverage (cấu hình qua biến `COVERAGE_THRESHOLD`)
- **Report level**: Sử dụng `tail -1` để lấy LINE counter cấp **report-level** (tổng hợp toàn module), không phải method-level
- **Exclude common-library**: Shared library từ upstream chỉ có 33% coverage, không phải service cần enforce
- **Parse**: Dùng `grep + sed` để extract attributes từ JaCoCo XML (thay thế awk bị lỗi parsing

ở phiên bản cũ)

- **Graceful skip**: Nếu module không có executable code hoặc report thiếu, pipeline không fail mà warning và skip



Hình 30: Console output hiển thị kiểm tra ngưỡng Coverage Gate ($\geq 70\%$)

9 Gitleaks – Secret Scanning

9.1 Vấn đề

Repository YAS chứa nhiều file cấu hình Kubernetes và Docker từ upstream, trong đó có các chuỗi base64-encoded trông giống API keys hoặc secrets. Cần phân biệt giữa **secrets thật sự mới** (do nhóm vô tình commit) và **false positives** từ upstream.

9.2 Công cụ: Gitleaks v8.18.4

Gitleaks là công cụ mã nguồn mở chuyên quét secrets trong source code. Nhóm sử dụng phiên bản 8.18.4 với chế độ `-no-git` (quét file trực tiếp, không cần git history).

9.3 Cấu hình

9.3.1 gitleaks.toml

File cấu hình chính, extend từ bộ rules mặc định và thêm allowlist cho các đường dẫn an toàn:

```
[extend]
useDefault = true

[allowlist]
paths = [
    '''k8s/.*''',
    '''docker/.*''',
    '''automation-ui/.*''',
    '''.github/.*''',
    '''scripts/.*''',
    '''.*\.md$''',
    '''.*\.json$''',
    '''gitleaks-baseline\.json''',
    '''gitleaks-report\.json'''
]
```

Listing 12: gitleaks.toml

9.3.2 .gitleaksignore

File chứa danh sách 13 fingerprints cụ thể của các secrets từ upstream cần bỏ qua:

```
# Upstream K8s secrets
6abd7e0e...  # k8s/base secrets
a7f12cb3...  # docker compose credentials
# ... (13 entries)
```

Listing 13: .gitleaksignore (trích)

9.3.3 gitleaks-baseline.json

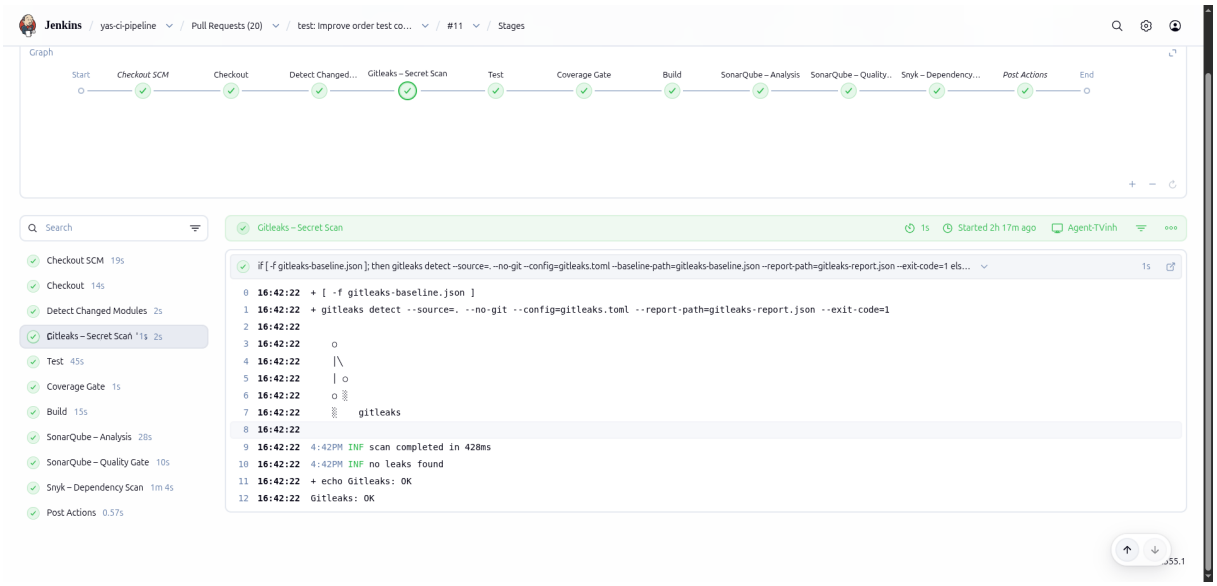
Baseline trống (`[]`), đảm bảo chỉ phát hiện secrets **mới** so với baseline.

9.4 Luồng quét trong Pipeline

1. Tải Gitleaks binary (cache nếu đã có)
2. Kiểm tra baseline file tồn tại
3. Chạy quét với `-no-git -config=gitleaks.toml -baseline-path=gitleaks-baseline.json`
4. Nếu phát hiện secret mới → **Pipeline FAIL** (`-exit-code=1`)
5. Nếu không phát hiện → In "Gitleaks scan PASSED"

9.5 Kết quả

- Tất cả các build đều pass Gitleaks scan
- 13 false positives từ upstream đã được xử lý qua `.gitleaksignore`
- Console output: `>> Gitleaks scan PASSED - no new secrets found`



Hình 31: Console output hiển thị Gitleaks quét thành công (không phát hiện rò rỉ secret)

10 SonarQube – Code Quality Analysis

10.1 Giới thiệu

SonarQube là nền tảng phân tích chất lượng code, phát hiện bugs, code smells, security vulnerabilities và đo lường coverage.

Nhóm sử dụng SonarQube **self-hosted** trên cùng AWS EC2 instance với Jenkins, truy cập tại `http://3.27.92.213:9000`.

10.2 Cấu hình Server

- **SonarQube URL**: `http://3.27.92.213:9000`
- **Authentication**: Token-based, lưu trong Jenkins Credentials (`sonarqube-token`)
- **Project Key**: `nashtech-garage_yas-yas-parent` (auto-detected từ pom.xml)

10.3 Tích hợp trong Pipeline

```
stage('SonarQube - Analysis') {
    steps {
        withSonarQubeEnv('sonar-server') {
            withCredentials([string(credentialsId: 'sonarqube-token', variable:
    'SONAR_TOKEN')]) {
                sh "mvn sonar:sonar -pl ${plArg} -am -Dsonar.host.url=..."
            }
        }
    }
}
stage('SonarQube - Quality Gate') {
    steps {
        withSonarQubeEnv('sonar-server') {
            timeout(time: 15, unit: 'MINUTES') {
                waitForQualityGate abortPipeline: false
            }
        }
    }
}
```

Listing 14: SonarQube Analysis & Quality Gate

10.4 Thiết kế và Quality Gate

- Chỉ phân tích **service modules** thay đổi (exclude common-library)

- Sử dụng `withSonarQubeEnv` để Jenkins tự động nhận biết cấu hình server.
- Stage **Quality Gate** sử dụng lệnh `waitForQualityGate` để treo Pipeline cho đến khi SonarQube phân tích xong và gửi webhook kết quả về lại Jenkins.

10.5 Kết quả

- Console: `[INFO] ANALYSIS SUCCESSFUL`
- SonarQube task URL được in ra để truy cập dashboard
- Dashboard hiển thị: bugs, vulnerabilities, code smells, coverage percentage, duplications

11 Snyk – Dependency Vulnerability Scan

11.1 Giới thiệu

Snyk là công cụ quét lỗ hổng bảo mật trong dependencies (thư viện phụ thuộc). Nhóm tích hợp Snyk CLI vào pipeline để tự động phát hiện các thư viện có CVE (Common Vulnerabilities and Exposures).

11.2 Cấu hình

- **Snyk CLI**: Cài đặt trên Jenkins server
- **Authentication**: Token-based, lưu trong Jenkins Credentials (`snyk-token`)
- **Severity threshold**: `high` — chỉ báo cáo lỗ hổng mức high trở lên

11.3 Tích hợp trong Pipeline

Stage Snyk thực hiện 2 bước cho mỗi service module:

11.3.1 Snyk Test

Quét dependencies và báo cáo lỗ hổng:

```
snyk test \
  --file=${module}/pom.xml \
  --severity-threshold=high \
```

```
  --all-sub-projects
```

Listing 15: Snyk Test command

11.3.2 Snyk Monitor

Đẩy snapshot dependencies lên Snyk dashboard để theo dõi liên tục:

```
snyk monitor \
  --file=${module}/pom.xml \
  --project-name=yas-${module}
```

Listing 16: Snyk Monitor command

11.4 Xử lý lỗi

Pipeline được thiết kế với cơ chế **retry** và **graceful degradation**:

1. Chạy `snyk test` lần 1
2. Nếu fail → retry với `-max-depth=3` (giảm độ sâu dependency tree)
3. Nếu vẫn fail → ghi warning, pipeline **tiếp tục** (không block)

11.5 Khắc phục lỗi phân giải Dependency (SNYK-CLI-0000)

Trong quá trình triển khai, nhóm từng gặp lỗi Snyk crash với mã `exit code: -13` (SNYK-CLI-0000) khi chạy trên môi trường Jenkins.

Nguyên nhân: Snyk nội bộ sử dụng `mvn dependency:tree` để phân tích. Với kiến trúc Monorepo, việc quét một module con (ví dụ `order`) bằng cờ `-f order/pom.xml` sẽ khiến Maven không tìm thấy file POM cha (`yas` parent) để nội suy biến `${revision}`, dẫn đến việc tiến trình cấp dưới của Snyk bị lỗi và trả về mã -13.

Cách giải quyết: Nhóm đã cấu hình lại Pipeline để chạy lệnh `mvn dependency:tree -pl ${module} -am` trước bước chạy Snyk. Lệnh này giúp Maven tải và cache sẵn toàn bộ dependency tree của module và POM cha vào local repository. Nhờ vậy, bước quét Snyk sau đó diễn ra hoàn toàn trơn tru và không còn lỗi.

12 Kết quả thực thi Pipeline

12.1 Danh sách Pull Requests

Bảng 7: Danh sách Pull Requests trên GitHub

PR	Title	Branch	Trạng thái
#5	ci: setup Jenkins pipeline	ci/setup-jenkins-pipeline	Merged
#6	test(tax): add coverage	tri/tax-tests	Open (CI pass)
#7	test(media): add coverage	feature/media-tests	Open (CI pass)
#8	fix(ci): coverage gate bug	ci/setup-jenkins-pipeline	Open

12.2 Kết quả Pipeline cho PR #7 (media)

Bảng 8: Kết quả pipeline PR #7 - Module media

Stage	Kết quả	Thời gian
Checkout SCM	PASS	7s
Tool Install	PASS	0.22s
Checkout	PASS	4s
Detect Changed Modules	PASS	0.74s
Gitleaks – Secret Scan	PASS	2s
Test	PASS (36 tests, 0 failures)	74s
Coverage Gate	PASS	—
Build	PASS	82ms
SonarQube – Analysis	PASS	10min
Snyk – Dependency Scan	Warning (exit -13)	45s

12.3 Kết quả Pipeline cho PR #6 (tax)

Bảng 9: Kết quả pipeline PR #6 - Module tax

Stage	Kết quả	Thời gian
Checkout SCM	PASS	—
Detect Changed Modules	PASS	—
Gitleaks – Secret Scan	PASS	—
Test	PASS (47 tests, 0 failures)	—
Coverage Gate	PASS	—
Build	PASS	—
SonarQube – Analysis	PASS	—
Snyk – Dependency Scan	Warning (exit -13)	—

12.4 Minh chứng Branch Protection hoạt động

Trên mỗi Pull Request, GitHub hiển thị:

1. **Review required**: "At least 2 approving reviews are required"
2. **Status checks**: CI pipeline phải pass trước khi merge
3. **Merge blocked**: Nút "Merge" bị disabled khi chưa đủ conditions

13 Phân công công việc

13.1 Thành viên nhóm

Bảng 10: Thông tin thành viên nhóm

Họ tên	**MSSV**	**Vai trò**
Nguyễn Lê Thế Vinh	[MSSV1]	Leader
Phạm Quang Vinh	[MSSV2]	Infra & Security
Lê Xuân Trí	[MSSV3]	Testing

13.2 Chi tiết phân công

13.2.1 Nguyễn Lê Thế Vinh (Leader)

- Fork repository, thêm collaborators
- Viết `Jenkinsfile` (8 stages, 298 dòng)
- Viết `ci/detect-changed-modules.sh` (monorepo path filter)
- Viết `ci/check-coverage.sh` (coverage gate $\geq$ 70%)
- Cấu hình `gitleaks.toml`, `.gitleaksignore`, `gitleaks-baseline.json`
- Cấu hình GitHub Rulesets (branch protection)
- Debug và fix awk parsing bug trong coverage gate
- Viết báo cáo
- Review và approve PRs

13.2.2 Phạm Quang Vinh (Infra & Security)

- Setup AWS EC2 instance

- Cài đặt và cấu hình Jenkins server
- Cài đặt và cấu hình SonarQube server
- Cài đặt Snyk CLI trên Jenkins server
- Cấu hình Jenkins Credentials (SonarQube token, Snyk token)
- Cấu hình Jenkins Global Tools (JDK-25, Maven)
- Tạo Multibranch Pipeline job
- Cấu hình Webhook GitHub → Jenkins
- Review và approve PRs

13.2.3 Lê Xuân Trí (Testing)

- Viết unit tests cho module **media** (PR #7)
  - MediaServiceUnitTest
  - FileSystemRepositoryTest
- Viết unit tests cho module **tax** (PR #6)
  - TaxRateServiceTest
  - TaxClassServiceTest
  - TaxRateControllerTest
- Đảm bảo coverage $\geq$ 70% cho từng module
- Tạo Pull Requests theo quy trình
- Review và approve PRs

13.3 Tổng kết đóng góp

Bảng 11: Tổng kết đóng góp theo thành viên

Hạng mục	**Vinh.NL**	**Vinh.PQ**	**Trí**
Pipeline (Jenkinsfile)	✓		
CI Scripts	✓		
Security Config	✓	✓	
Infrastructure		✓	
Jenkins Setup		✓	
SonarQube Setup		✓	
Unit Tests – media			✓
Unit Tests – tax			✓
Branch Protection	✓		
Code Review	✓	✓	✓
Báo cáo	✓		

14 Kết luận

14.1 Tổng kết

Nhóm đã hoàn thành việc xây dựng hệ thống CI cho dự án YAS (Yet Another Shop) với đầy đủ các yêu cầu:

Bảng 12: Checklist yêu cầu đồ án

STT	**Yêu cầu**	**Trạng thái**
1	Sử dụng Jenkins	✓
2	Fork repo YAS	✓
3	Branch protection (2 approvals + CI pass)	✓
4	Jenkins quét từng branch	✓
5	Pipeline $\geq$ 2 phases (test + build) + coverage	✓
6	Monorepo path filter	✓
7a	Thêm unit test cho $\geq$ 2 services	✓
7b	Coverage gate $\geq$ 70%	✓
7c	Security scanning (Gitleaks + SonarQube + Snyk)	✓

14.2 Điểm nổi bật

1. **Pipeline 8 stages**: Thiết kế pipeline đầy đủ từ checkout đến security scanning, mỗi stage có mục đích rõ ràng

2. **Monorepo-aware**: Path filter thông minh giúp giảm thời gian build từ ∼30 phút xuống ∼5 phút
3. **Coverage Gate thực sự enforce**: Parse JaCoCo XML report và so sánh với ngưỡng 70%, pipeline FAIL nếu không đạt
4. **Gitleaks với baseline**: Xử lý 13 false positives từ upstream bằng `.gitleaksignore` và baseline, chỉ phát hiện secrets mới
5. **Retry mechanism cho Snyk**: Tự động retry với `-max-depth=3` nếu lần quét đầu fail
6. **Graceful degradation**: Pipeline không bị block bởi các lỗi infrastructure (Snyk OOM, shallow clone), nhưng vẫn báo warning

14.3 Khó khăn gặp phải

1. **Shallow clone**: Jenkins Multibranch Pipeline mặc định dùng shallow clone, khiến `git diff` không hoạt động. Fix bằng `git fetch origin main`.
2. **Upstream secrets**: Repository gốc chứa nhiều K8s secrets dạng base64, gây false positives cho Gitleaks. Fix bằng `.gitleaksignore` + `gitleaks.toml` allowlist.
3. **Awk parsing bug**: Script coverage gate ban đầu dùng awk nhưng logic so sánh field không đúng, dẫn đến coverage gate luôn skip. Fix bằng `grep + sed`.
4. **Snyk OOM**: EC2 instance thiếu RAM khiến Snyk CLI bị kill (exit -13). Workaround bằng retry + graceful degradation.
5. **Jenkins branch status pending**: Multibranch Pipeline disable branch builds khi branch đã có PR, khiến status "branch" kẹt pending trên GitHub. Đây là known behavior, không ảnh hưởng merge.

14.4 Hướng phát triển

- Thêm **CD (Continuous Deployment)**: Tự động deploy lên staging/production
- Tăng coverage cho **tất cả service modules**, không chỉ media và tax
- Fix Snyk OOM bằng cách nâng cấp EC2 instance
- Thêm **DAST** (Dynamic Application Security Testing) cho các API endpoints
- Tích hợp **Slack/Discord notification** khi pipeline fail

Tài liệu

[1] NashTech Garage, "YAS: Yet Another Shop," https://github.com/nashtech-garage/yas

[2] Jenkins, "Jenkins User Documentation," https://www.jenkins.io/doc/

[3] Gitleaks, "Gitleaks - Protect and discover secrets," https://github.com/gitleaks/gitleaks

[4] SonarSource, "SonarQube Documentation," https://docs.sonarqube.org/latest/

[5] Snyk, "Snyk CLI Documentation," https://docs.snyk.io/snyk-cli

[6] EclEmma, "JaCoCo Java Code Coverage Library," https://www.jacoco.org/jacoco/

[7] Jenkins, "Pipeline Multibranch," https://www.jenkins.io/doc/book/pipeline/multibranch/